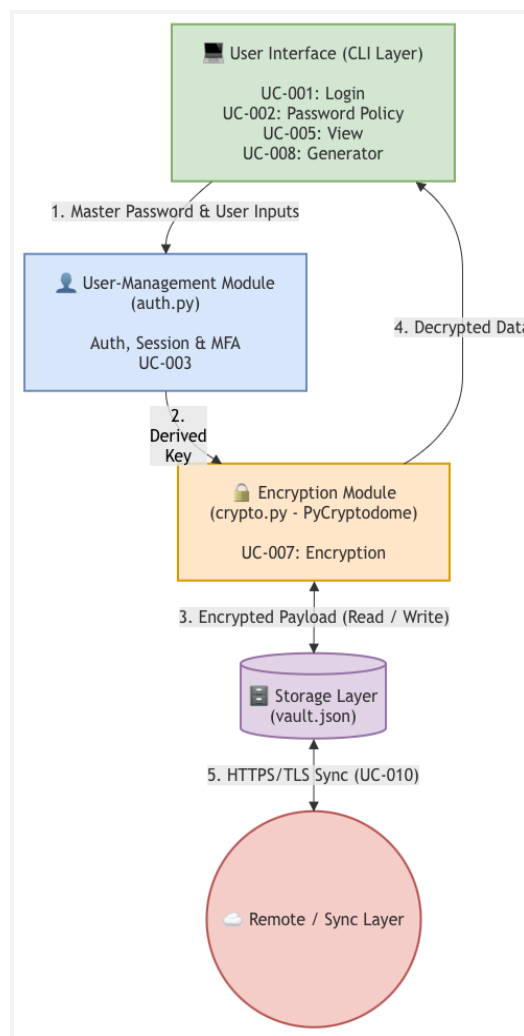


Section 1: System Architecture & Data Flow

1.1 Architectural Overview

The system is designed with a modular architecture that separates user interaction, authentication, cryptographic operations, and data persistence. This separation of concerns ensures that sensitive data in plaintext is strictly confined to necessary modules and its footprint is minimized in the system's memory.



1.2 Component Responsibilities

Component	Core Responsibilities	Associated Use Cases
User Interface (CLI Layer)	Handles all command-line interactions. Prompts users for input, displays results, and enforces input validation (like strong password policies) before passing data downstream.	UC-001 (Login), UC-002 (Password Policy), UC-004 (Store Password), UC-005 (View), UC-006 (Update), UC-008 (Generate Strong Password)
User-Management Module (auth.py)	Captures the master password securely. Manages user session state, implements session timeouts, and handles optional multi-factor authentication (MFA) workflows. Triggers the derivation of the master key.	UC-003 (Multi-factor authentication)
Encryption Module (crypto.py)	The cryptographic engine utilizing PyCryptodome. Derives encryption keys using robust Key Derivation Functions (scrypt) and performs authenticated encryption/decryption (AES-256-GCM) on the password database.	UC-007 (Encrypt password database)
Storage Layer (storage.py)	Manages local file I/O operations. Reads and writes the encrypted payload and necessary plaintext metadata (salt, nonce, tag)	All local persistence-related flows

Component	Core Responsibilities	Associated Use Cases
	to a local JSON file (vault.json).	

1.3 Standard Data Flow

- User Input & Authentication:** The user interacts with the CLI to log in. The User-Management module securely captures the Master Password (UC-001).
- Key Derivation:** The plaintext Master Password is sent to the Encryption Module, which uses a secure KDF (scrypt) along with a salt to derive a 256-bit symmetric encryption key.
- Data Retrieval & Decryption:** The Storage Layer reads vault.json. The Encryption Module uses the derived key to authenticate and decrypt the ciphertext (AES-256-GCM).
- In-Memory Processing:** The decrypted database resides temporarily in memory. The user performs actions (View, Add, Update - UC-004, UC-005, UC-006). Plaintext data lifetimes are minimized.
- Encryption & Storage:** Upon saving new changes, the Encryption Module encrypts the updated database (UC-007) generating a new authentication tag. The Storage Layer overwrites vault.json.

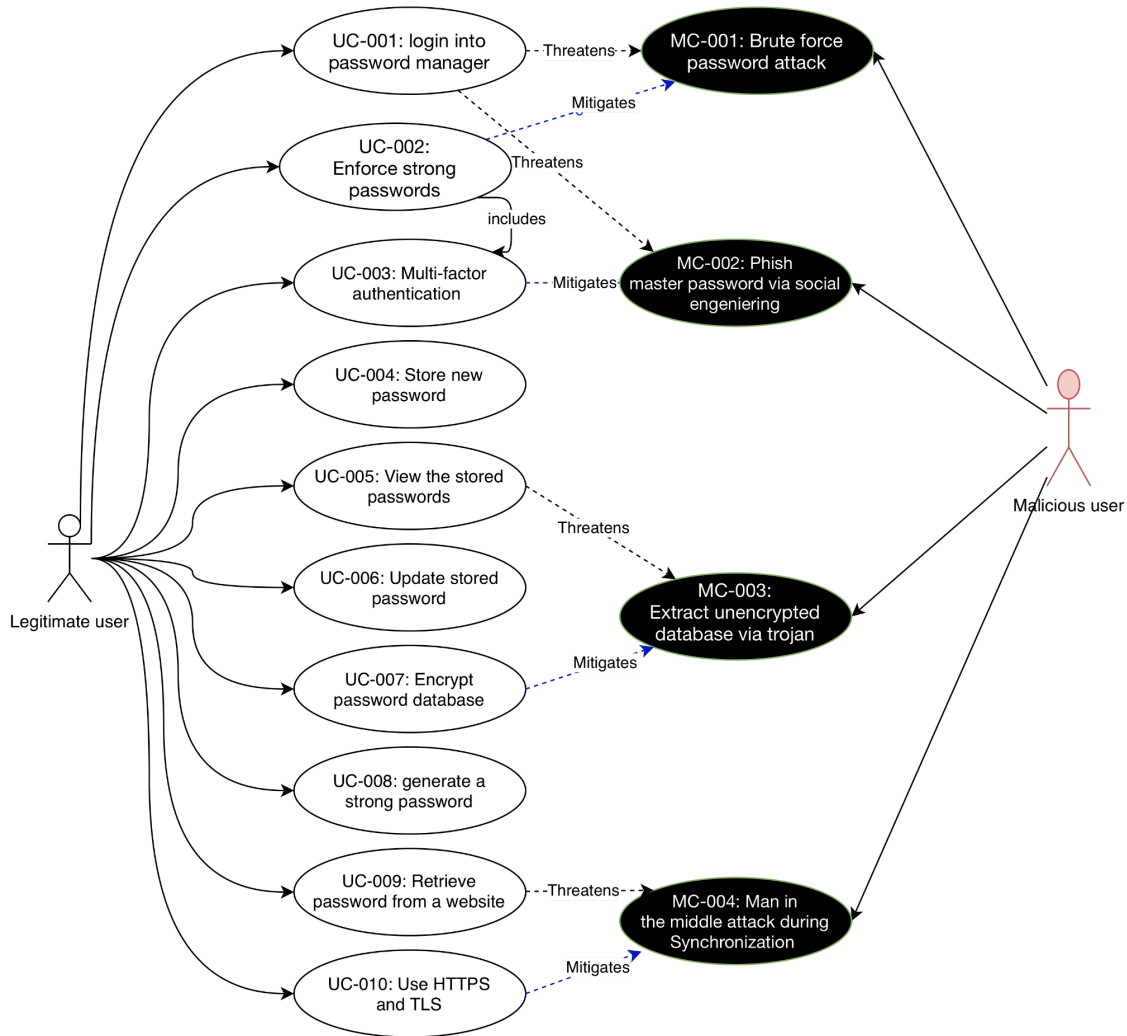
Section 2: Threat Model and Mitigations

2.1 Identified Threats and Mitigations

Misuse Case (Threat)	Target / Trigger	Impact	Mitigation Strategy
MC-001: Brute force password attack	UC-001: Master Password (Offline cracking)	High	UC-002: Enforce strong master passwords combined with slow

Misuse Case (Threat)	Target / Trigger	Impact	Mitigation Strategy
			Key Derivation Functions (scrypt with $N=2^{14}$, $r=8$, $p=1$) and unique 16-byte salts.
MC-002: Phish master password via social engineering	Master password capture	Critical	UC-003: Implement Multi-Factor Authentication (MFA). A compromised master password alone will not grant vault access.
MC-003: Extract unencrypted database via trojan	UC-005: Vault at Rest (Disk files)	Critical	UC-007: Encrypt the password database using AES-256-GCM . Data on disk remains fully encrypted at rest, preventing malware from reading raw secrets.
MC-004: Man in the middle attack during synchronization	UC-009: Vault in Transit (Sync)	High	UC-010: Enforce HTTPS and TLS encryption for all network communication and remote synchronization to prevent eavesdropping.

Misuse Case (Threat)	Target / Trigger	Impact	Mitigation Strategy
Memory Extraction (RAM Scraping)	Vault in Memory	High	Keep plaintext lifetime minimal. Use mutable byte arrays (bytearray) to explicitly zero-out keys and buffers immediately after use.
Shoulder Surfing / Terminal History	Interface (CLI)	Medium	Mask input during password entry using <code>getpass.getpass()</code> . Accept sensitive parameters only via interactive prompts, never via command-line arguments.



Section 3: Vault Format and Cryptographic Scheme

3.1 Cryptographic Primitives

- **Cryptographic Engine:** PyCryptodome module for Python.
- **Cipher Strategy:** AES-256-GCM (Galois/Counter Mode) for robust Authenticated Encryption with Associated Data (AEAD).
- **Key Derivation Function (KDF):** scrypt configuration (N=16384, r=8, p=1) to prevent brute-force and GPU-based dictionary attacks.

- **Entropy:** `Crypto.Random.get_random_bytes` (OS-level CSPRNG) utilized for salt, nonce, and password generation.

3.2 On-Disk Vault Schema (vault.json)

The vault is stored locally as a JSON file. Metadata necessary for decryption is stored in Base64 encoding. The actual credential records are serialized into a JSON string and fully encrypted before being written to the ciphertext field.

On the repository

Section 4: Repository Initialization

On the repository